

PESTO: Parametric Execution-path-Sensitivity for Test Optimization.

Seokhun Jeong, Jeongwon Park, Eunsu Kim, and Jihong Min

 **GitHub:** github.com/KunJeong/pesto

1 Introduction

Interpreters are critical testing targets in software engineering. Ensuring their correctness is crucial because a bug in an interpreter can compromise all applications running on top of it. For this reason, verifying the compliance and robustness of interpreters has remained a major focus of software engineering research.

In general software testing, a negative test is designed to verify how a system handles invalid inputs, typically expecting the oracle to fail. However, for a meta-program like an interpreter that accepts a program as input, an input code that terminates with an error is conventionally referred to as a negative test case. Strictly speaking, gracefully raising an exception is a correct output from the interpreter’s perspective. Nevertheless, in this study, we adopt this idiomatic sense and define the collection of error-inducing programs as our negative test suite.

Implementing a negative test suite is essential for verifying an interpreter’s error-handling mechanisms. It directly validates whether the interpreter can properly detect, reject, and process faulty inputs. Without such a test suite, interpreters remain vulnerable to runtime failures, when encountering malformed programs.

Having a well-designed set of negative tests lets you ensure existing error-handling behavior is preserved. When multiple implementations exist, it can also ensure that these implementations agree on input validity. A well-designed set would preferably achieve this with a small number of tests, and a short total execution time. Thus, an optimal negative test suite is one that sufficiently covers diverse invalid conditions with a small number of minimal programs.

To construct well-designed negative test suites, automated test generation techniques are widely used, yet they inherently introduce significant redundancy. Since these automated techniques, such as fuzzing, randomly or systematically generate test inputs to maximize coverage, they frequently generate a large volume of overlapping test cases that target the exact same internal execution behavior. Executing such a redundant test suite incurs unnecessary computational costs, degrading the efficiency of the software testing process. This redundancy becomes particularly inefficient because the code locations that raise exceptions within an interpreter are limited.

To mitigate this redundancy, delta-debugging [1], [2] can be used to minimize invalid programs while ensuring that the same error behavior is preserved. Unlike positive tests, each negative test has exactly one reason it is invalid. Delta-debugging can be used to reduce invalid programs until the source of failure is isolated. To reduce a program, one first defines a property that must be preserved for the duration of the reduction. In the case of negative tests, the exception type or the error message can be the guiding property.

However, the same exception type and error message can be triggered by syntactically different input programs. In the case of CPython, both `1+'x'` and `sum('z', 3)` raise a `TypeError: unsupported operand type(s) for +: 'int' and 'str'`, despite executing distinct internal paths. This means that the exception type and error message alone are too coarse-grained to preserve the diversity of the test suite. Therefore, we need additional information to guide the reducer, so that the resulting test suite is small and each program minimal, yet retains the original diversity.

To address this challenge, we propose PESTO (Parametric Execution-path-Sensitivity for Test Optimization), a framework designed to achieve a well-designed negative test suite by balancing test-suite minimality and execution-path diversity. PESTO preserves a configurable prefix depth of the internal call trace during program reduction. Our evaluation on CPython shows that a low sensitivity level ($k=2$) preserves 99.7% of the mutation-detection capability of the maximum sensitivity baseline, while producing highly compact negative tests.

2 Implementation

2.1 Overview

The overall structure of **PESTO** consists of four main components: seed generation using grammar-based fuzzing, trace instrumentation of the interpreter, trace-sensitive reduction, and mutation testing.

First, we use grammar-based fuzzing to generate well-formed Python programs that trigger runtime exceptions. Next, we instrument CPython to extract the failure traces of each input program. Using this trace information, we reduce Python programs while preserving both their exception types and failure traces, where the amount of trace information to preserve is controlled by a sensitivity parameter. Finally, we evaluate the reduced set of seed programs using mutation testing.

2.2 Grammar-Based Fuzzing

We utilize Grammarinator [3] for the grammar-based fuzzing stage. In this stage, the fuzzer first generates candidate Python programs based on our grammar specifications. We then execute these inputs and retain programs that trigger runtime exceptions, discarding both normal executions and syntax errors.

Using the full Python3 specification is ineffective for generating negative test cases. It primarily produces programs that either fail with a `SyntaxError` or hold no testing value, such as halting due to an unknown import, leaving few valid runtime-exception-inducing inputs.

We therefore define a customized grammar that generates syntactically valid programs by mixing domain-specific code fragments with error-triggering operations. The grammar covers common Python features such as arithmetic, strings, collections, objects, byte sequences, type conversions, paths, and dequeues, allowing the fuzzer to produce inputs that are more likely to reach runtime exception paths rather than fail during parsing.

Given a random seed and a target number of negative tests, our fuzzing implementation repeatedly generates and executes candidate programs until the target number of runtime-exception-inducing inputs is reached. We generated the baseline negative test suite consisting of 2,000 tests using seed 42, and the generation completed within 10 seconds. The generated suite covers several runtime exception types, including `TypeError`, `ValueError`, `IndexError`, `UnboundLocalError`, `ZeroDivisionError`, `StopIteration`, `OverflowError`, `KeyError`, and `AttributeError`.

2.3 Trace Instrumentation

Trace instrumentation proceeds in two stages. First, the CPython source is patched and rebuilt to print a backtrace on exceptions. Second, the backtrace is filtered to obtain the backtrace corresponding to the user-facing exception.

Instrumentation inserts a function `pesto_emit` that takes a Python exception and prints the backtrace using the C standard library functions `backtrace` and `backtrace_symbols_fd`. To facilitate filtering the backtrace by exception type in step 2, the originating exception type is added to the beginning of the backtrace block. The instrumentation patch is saved as a git `.patch` file so that it can be applied consistently to new clones of the CPython source.

We need additional regular-expression parsing and filtering logic on the Python wrapper to filter exactly the exception that we want. Because Python uses exceptions for internal control-flow, the same `TypeError` exception is raised multiple times during a single run. Python uses the `PyErr_Display` function when printing the exception to the user, so we can filter the backtrace with this function to obtain the backtrace that corresponds to the exact user-facing exception.

2.4 Trace-sensitive Reduction

Given trace-sensitivity information from invalid Python programs, we use Perses [4], a state-of-the-art language-agnostic syntax-guided program reducer, to generate reduced programs that preserve both the exception type and the execution traces of their original input programs.

Most program reducers, including Perses, require two inputs: a program to be reduced and a test script that checks whether a candidate reduced program preserves the desired property of the original program. In our setting, the input to the reducer is an invalid Python program generated by our grammar-based fuzzer. The reducer then iteratively simplifies the program, accepting only those reductions that still satisfy the property checked by the accompanying test script.

However, preserving only the error type is too coarse-grained for our purpose. Two programs may raise the same type of exception while exercising different parts of the Python interpreter. If the reducer only checks whether the reduced program raises the same error type, it may aggressively remove code that is important for covering diverse interpreter execution paths. To avoid this, we collect execution traces from the seed programs and use them as an additional reduction criterion.

We parameterize the degree of trace sensitivity. A trace-sensitivity parameter controls how much of the original execution trace must be preserved during reduction. For example, if the trace-sensitivity parameter is set to 5 and the original Python program raises a `TypeError`, then the reduced program must also raise a `TypeError` and preserve the same call trace up to depth 5. The test script given to Perses encodes this property: a candidate program is accepted only if it preserves both the error type and the required prefix of the execution trace. By varying this parameter, we can identify a suitable level of trace sensitivity that keeps the test suite both compact and diverse.

2.5 Mutation Testing

Mutation testing is used to evaluate the performance of the generated test suite. We employ `pycparser` to modify source code and generate mutants. `pycparser` is a Python project that constructs abstract syntax trees (ASTs) from C programs, providing functionality analogous to Python’s `ast` module.

The mutation operators are based on the framework proposed by Agrawal et al. [5]. Specifically, we select ten fundamental mutation operators from Barbosa et al. [6]: relational operator replacement (ORRN), twiddle mutations (VTWD), domain traps (VDTR), arithmetic-operator-to-shift-operator replacement (OASN), logical-operator-to-bitwise-operator replacement (OLBN), `while-to-do-while` replacement (SWDD), statement deletion (SSDL), constant-for-constant replacement (CCRC), constant-for-scalar replacement (CCRS), and *n*-trip continue (SMTC).

When generating mutants, `PESTO` applies all ten mutation operators and produces three artifacts: (1) the mutated source code, (2) `pesto_runtime.c`, which uses the variable `__pesto_mutant_id` to determine at runtime which mutant is activated, and (3) `pesto.json`, which stores metadata describing the generated mutants.

A primary challenge in mutation testing is reducing its computational cost [7]. In principle, every test case in the test suite must be executed against every mutant. Since C is a compiled language, we embed all mutants into a single executable and use `__pesto_mutant_id` to select the active mutant at runtime. This approach avoids recompilation for each mutant and significantly reduces the overall execution cost.

After generating mutants and a test suite, `PESTO` evaluates the mutation score using the information stored in `pesto.json`. It first executes the test suite on the original program to obtain baseline outputs, then runs each mutant against the same tests. A mutant is considered killed if its behavior differs from the baseline, including differences in timeout status, exit code, standard output, or exception type. Evaluation stops as soon as a mutant is killed by a test case; otherwise, it is recorded as survived.

2.6 RQ1. Does Trace Sensitivity Reflect Test Diversity?

We evaluate the effectiveness of the extracted test suites using mutation testing. Although only a subset of mutation operators is employed, mutating the entire CPython codebase remains computationally expensive, as the repository has grown to approximately 2.9 million lines of code [8]. To address this issue, we first identify functions exercised by the test suite and construct `examples_cpython_targets.json`, which contains functions executed by at least one test case. Mutations are then generated only within these target functions. This restriction significantly

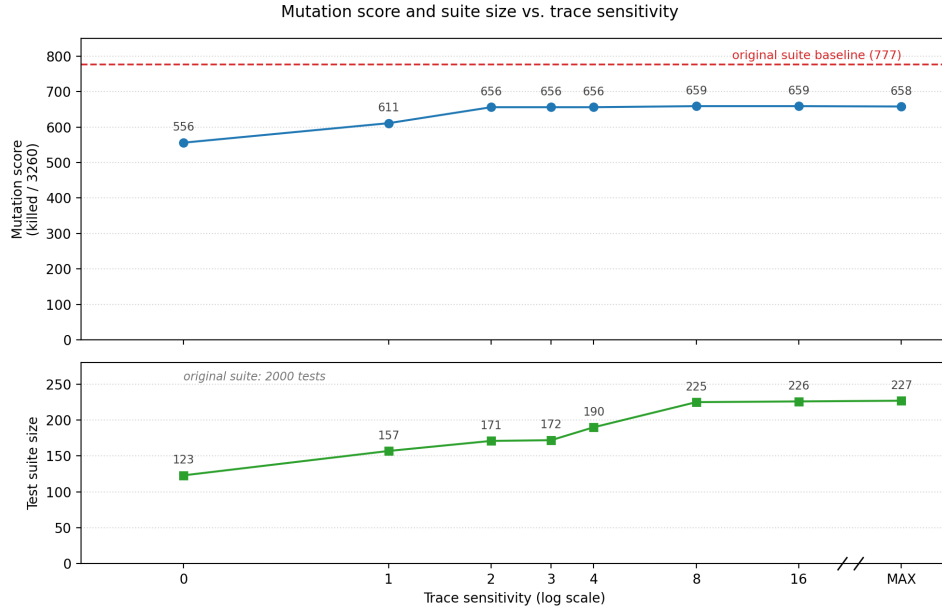


Figure 1: Effect of trace sensitivity on mutation score and number of test cases. Top: Mutation score, Bottom: Number of test cases.

reduces the computational cost while preserving all mutants that could potentially be killed by the evaluated test suites. Since a mutant can only be killed if the corresponding code is executed, mutations outside the covered functions cannot affect the mutation score.

Using this procedure, we generated 3,260 mutants. We then evaluated test suites extracted with sensitivity of 0, 1, 2, 3, 4, 8, 16, and ∞ . As a baseline, we also evaluated randomly sampled test suites of the same size at sensitivity 0. The results are shown in Figure 1.

Figure 1 shows that both mutation score and the number of retained test cases increase with trace sensitivity. The mutation score rises from 556 at sensitivity 0 to 656 at sensitivity 2, and then saturates, reaching 659 at sensitivity 8 and 16. Meanwhile, the number of retained test cases increases from 123 at sensitivity 0 to 227 at maximum sensitivity.

As the sensitivity threshold increases, both the number of selected test cases and the mutation score increase. This is expected, as higher sensitivity retains a larger and more diverse set of execution traces. However, the improvement quickly saturates. Notably, the test suite extracted with sensitivity 2 achieves 99.7% of the mutation score obtained with sensitivity ∞ , despite containing substantially fewer test cases. This suggests that relatively low sensitivity values are sufficient to capture most of the diversity relevant to mutation testing.

Interestingly, the randomly sampled baseline achieves a higher mutation score than the extracted test suite at sensitivity ∞ . A possible explanation is that the extracted test suites undergo a reduction process, which removes tests that are redundant for preserving the failure-inducing behavior. In contrast, the random baseline retains additional tests that may exercise unrelated program functionality, creating more opportunities to kill mutants. Therefore, this result does not necessarily imply that the extracted test suites are less diverse with respect to the target failure.

Overall, the results indicate a strong relationship between trace diversity, as controlled by the sensitivity parameter, and mutation-testing effectiveness. They also suggest that aggressive trace reduction can substantially decrease test-suite size while preserving nearly all mutation-detection capability.

2.7 RQ2. How Small Is the Trace-sensitive Reduced Test Suite?

Program size is an important metric for negative tests, since fault localization techniques such as delta debugging aim to minimize failure-inducing inputs so that they become easier to understand

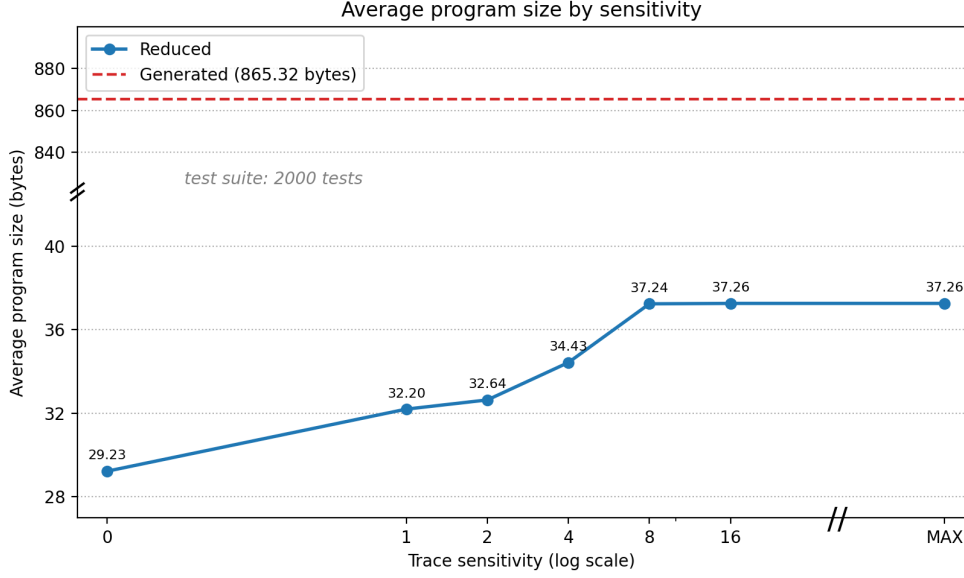


Figure 2: Average program size of the reduced test suite under different trace-sensitivity settings.

and debug. Therefore, we measure how small the trace-sensitive reduced test suite becomes under different sensitivity settings.

As shown in Figure 2, the original test suite consists of 2,000 tests, with an average program size of 865.32 bytes. We reduce each test in the suite using different trace-sensitivity settings: $s = 0, 1, 2, 4, 8, 16$, and MAX . When $s = 0$, the average program size is reduced to 29.23 bytes, corresponding to a 96.62% reduction. As the trace sensitivity increases, the average program size also increases monotonically. However, even at the maximum sensitivity setting, the average size remains only 37.26 bytes, which is still substantially smaller than the average size of the original test suite. This result shows that trace-sensitive reduction can preserve additional execution-trace information while still keeping the reduced tests highly compact.

2.8 RQ3. How Fast Is the Trace-sensitive Reduced Test Suite?

We also measure execution time as a minimality metric for negative test suites.

The results follow the same trend as the size results in RQ2: the generated baseline test suite takes 33.05 seconds to execute, whereas all trace-sensitive reduced suites complete within 3.5 seconds. Specifically, the execution times are 1.899, 2.403, 2.637, 2.896, 3.480, 3.479, and 3.479 seconds for sensitivity 0, 1, 2, 4, 8, 16, and maximum sensitivity, respectively. The execution time increases with sensitivity and saturates after sensitivity 8, matching the trend observed for test-suite size.

3 Conclusion

Negative tests for meta-programs such as interpreters are essential for validating whether invalid input programs are rejected through expected error-handling behavior. However, because the same exception type can arise from different interpreter execution paths, reducing negative tests only by their exception type may lose important behavioral diversity. In this paper, we present **PESTO**, a framework for optimizing negative test suites with parametric execution-path sensitivity. By preserving a configurable prefix of exception-triggering execution traces during reduction, **PESTO** controls the trade-off between test-suite minimality and execution-path diversity. Our evaluation on CPython shows that a low sensitivity level, with $k = 2$, already preserves 99.7% of the mutation-detection capability of the test suite reduced with maximum sensitivity, while producing substantially smaller and more concise negative tests.

References

- [1] A. Zeller and R. Hildebrandt, “Simplifying and isolating failure-inducing input”, *IEEE Transactions on Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002. DOI: 10.1109/32.988498.
- [2] G. Misherghi and Z. Su, “Hdd: Hierarchical delta debugging”, ser. ICSE ’06, Shanghai, China: Association for Computing Machinery, 2006, pp. 142–151, ISBN: 1595933751. DOI: 10.1145/1134285.1134307. [Online]. Available: <https://doi.org/10.1145/1134285.1134307>.
- [3] R. Hodován, Á. Kiss, and T. Gyimóthy, “Grammarinator: A grammar-based open source fuzzer”, in *Proceedings of the 9th ACM SIGSOFT International Workshop on Automating TEST Case Design, Selection, and Evaluation*, ser. A-TEST 2018, Lake Buena Vista, FL, USA: Association for Computing Machinery, 2018, pp. 45–48, ISBN: 9781450360531. DOI: 10.1145/3278186.3278193. [Online]. Available: <https://doi.org/10.1145/3278186.3278193>.
- [4] C. Sun, Y. Li, Q. Zhang, T. Gu, and Z. Su, “Perses: Syntax-guided program reduction”, in *Proceedings of the 40th International Conference on Software Engineering*, ser. ICSE ’18, Gothenburg, Sweden: Association for Computing Machinery, 2018, pp. 361–371, ISBN: 9781450356381. DOI: 10.1145/3180155.3180236. [Online]. Available: <https://doi.org/10.1145/3180155.3180236>.
- [5] H. Agrawal et al., “Design of mutant operators for the c programming language”, Technical Report SERC-TR-41-P, Software Engineering Research Center, Purdue . . . , Tech. Rep., 1989.
- [6] E. F. Barbosa, J. C. Maldonado, and A. M. R. Vincenzi, “Toward the determination of sufficient mutant operators for c”, *Software Testing, Verification and Reliability*, vol. 11, no. 2, pp. 113–136, 2001.
- [7] A. V. Pizzoleto, F. C. Ferrari, J. Offutt, L. Fernandes, and M. Ribeiro, “A systematic literature review of techniques and metrics to reduce the cost of mutation testing”, *Journal of Systems and Software*, vol. 157, p. 110388, 2019.
- [8] S. Ulbrych. “Cpython: 36 years of source code”. Python Insider (Official Python Core Development Team Blog), accessed 2026-06-07. [Online]. Available: <https://blog.python.org/2026/03/cpython-codebase-growth/>.